Import library

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
movies = pd.read_csv(r'C:\Users\user\Desktop\archive\movie.csv')
```

In [3]:
```
movies.head()
```

Out[3]:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

In [4]:
```
movies.tail()
```

Out[4]:

	movieId	title	genres
27273	131254	Kein Bund für's Leben (2007)	Comedy
27274	131256	Feuer, Eis & Dosenbier (2002)	Comedy
27275	131258	The Pirates (2014)	Adventure
27276	131260	Rentun Ruusu (2001)	(no genres listed)
27277	131262	Innocence (2014)	Adventure|Fantasy|Horror

In [5]:
```
movies.columns
```

Out[5]:
```
Index(['movieId', 'title', 'genres'], dtype='object')
```

In [6]:
```
movies.shape
```

Out[6]:
```
(27278, 3)
```

In [7]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27278 entries, 0 to 27277
Data columns (total 3 columns):
 #   Column   Non-Null Count  Dtype 
---  ------   --------------  ----- 
 0   movieId  27278 non-null  int64 
 1   title    27278 non-null  object
 2   genres   27278 non-null  object
dtypes: int64(1), object(2)
memory usage: 639.5+ KB
```

In [8]:
```
tags = pd.read_csv(r'C:\Users\user\Desktop\archive\tag.csv')
```

In [9]:
```
tags.shape
```

Out[9]: (465564, 4)

In [10]:
```
tags.info
```

Out[10]:
```
<bound method DataFrame.info of         userId  movieId           tag            timestamp
0           18     4141    Mark Waters  2009-04-24 18:19:40
1           65      208      dark hero  2013-05-10 01:41:18
2           65      353      dark hero  2013-05-10 01:41:19
3           65      521  noir thriller  2013-05-10 01:39:43
4           65      592      dark hero  2013-05-10 01:41:18
...        ...      ...            ...                  ...
465559  138446    55999        dragged  2013-01-23 23:29:32
465560  138446    55999  Jason Bateman  2013-01-23 23:29:38
465561  138446    55999         quirky  2013-01-23 23:29:38
465562  138446    55999            sad  2013-01-23 23:29:32
465563  138472      923  rise to power  2007-11-02 21:12:47

[465564 rows x 4 columns]>
```

In [11]:
```
tags.head()
```

Out[11]:

	userId	movieId	tag	timestamp
0	18	4141	Mark Waters	2009-04-24 18:19:40
1	65	208	dark hero	2013-05-10 01:41:18
2	65	353	dark hero	2013-05-10 01:41:19
3	65	521	noir thriller	2013-05-10 01:39:43
4	65	592	dark hero	2013-05-10 01:41:18

In [12]:
```
tags.columns          # it displays name of the columns
```

Out[12]:
```
Index(['userId', 'movieId', 'tag', 'timestamp'], dtype='object')
```

In [13]:
```
ratings = pd.read_csv(r'C:\Users\user\Desktop\archive\rating.csv')
```

In [14]:
```
ratings.shape
```

Out[14]: (20000263, 4)

In [15]:
```
ratings.columns
```

Out[15]:
```
Index(['userId', 'movieId', 'rating', 'timestamp'], dtype='object')
```

In [16]:
```
ratings.head()
```

Out[16]:

	userId	movieId	rating	timestamp
0	1	2	3.5	2005-04-02 23:53:47
1	1	29	3.5	2005-04-02 23:31:16
2	1	32	3.5	2005-04-02 23:33:39
3	1	47	3.5	2005-04-02 23:32:07
4	1	50	3.5	2005-04-02 23:29:40

In [17]:
```
ratings.tail
```

Out[17]:
```
<bound method NDFrame.tail of           userId  movieId  rating            timestamp
0              1        2     3.5  2005-04-02 23:53:47
1              1       29     3.5  2005-04-02 23:31:16
2              1       32     3.5  2005-04-02 23:33:39
3              1       47     3.5  2005-04-02 23:32:07
4              1       50     3.5  2005-04-02 23:29:40
...          ...      ...     ...                  ...
20000258  138493    68954     4.5  2009-11-13 15:42:00
20000259  138493    69526     4.5  2009-12-03 18:31:48
20000260  138493    69644     3.0  2009-12-07 18:10:57
20000261  138493    70286     5.0  2009-11-13 15:42:24
20000262  138493    71619     2.5  2009-10-17 20:25:36

[20000263 rows x 4 columns]>
```

In [23]:
```
del ratings['timestamp']
del tags['timestamp']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, ke
y)
   3811 try:
-> 3812     return self._engine.get_loc(casted_key)
   3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjectHashTabl
e.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.PyObjectHashTabl
e.get_item()

KeyError: 'timestamp'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[23], line 1
----> 1 del ratings[           ]
      2 del tags['timestamp']

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:4528, in NDFrame.__delitem__(self, k
ey)
   4523             deleted = True
   4524 if not deleted:
   4525     # If the above loop ran and didn't delete anything because
   4526     # there was no match, this call should raise the appropriate
   4527     # exception:
-> 4528     loc = self.axes[-1].get_loc(key)
   4529     self._mgr = self._mgr.idelete(loc)
   4531 # delete from the caches

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3819, in Index.get_loc(self, ke
y)
   3814     if isinstance(casted_key, slice) or (
   3815         isinstance(casted_key, abc.Iterable)
   3816         and any(isinstance(x, slice) for x in casted_key)
   3817     ):
   3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
   3820 except TypeError:
   3821     # If we have a listlike key, _check_indexing_error will raise
   3822     #  InvalidIndexError. Otherwise we fall through and re-raise
   3823     #  the TypeError.
   3824     self._check_indexing_error(key)

KeyError: 'timestamp'
```

Series

In [24]:
```
row_0 = tags.iloc[0]  # series
type(row_0)
```

Out[24]: pandas.core.series.Series

In [25]:
```
print(row_0)
```

```
userId                      18
movieId                   4141
tag                Mark Waters
timestamp    2009-04-24 18:19:40
Name: 0, dtype: object
```

In [26]:
```
row_0.index
```

Out[26]:
```
Index(['userId', 'movieId', 'tag', 'timestamp'], dtype='object')
```

In [27]:
```
row_0['userId']
```

Out[27]:
```
np.int64(18)
```

In [28]:
```
'ratings' in row_0
```

Out[28]:
```
False
```

In [29]:
```
row_0.name
```

Out[29]:
```
0
```

In [30]:
```
row_0=row_0.rename('firstRow')
row_0.name
```

Out[30]:
```
'firstRow'
```

Dataframes

In [31]:
```
tags.head()
```

Out[31]:

	userId	movieId	tag	timestamp
0	18	4141	Mark Waters	2009-04-24 18:19:40
1	65	208	dark hero	2013-05-10 01:41:18
2	65	353	dark hero	2013-05-10 01:41:19
3	65	521	noir thriller	2013-05-10 01:39:43
4	65	592	dark hero	2013-05-10 01:41:18

In [32]:
```
tags.index
```

Out[32]:
```
RangeIndex(start=0, stop=465564, step=1)
```

In [ ]:
```
tags.columns
```

In [ ]:
```
tags.iloc[[0,11,500]]
```

Descriptive Statistics

In [33]:
```
ratings['rating'].describe()
```

Out[33]:
```
count    2.000026e+07
mean     3.525529e+00
std      1.051989e+00
min      5.000000e-01
25%      3.000000e+00
50%      3.500000e+00
75%      4.000000e+00
max      5.000000e+00
Name: rating, dtype: float64
```

In [34]:
```
ratings.describe()
```

Out[34]:

	userId	movieId	rating
count	2.000026e+07	2.000026e+07	2.000026e+07
mean	6.904587e+04	9.041567e+03	3.525529e+00
std	4.003863e+04	1.978948e+04	1.051989e+00
min	1.000000e+00	1.000000e+00	5.000000e-01
25%	3.439500e+04	9.020000e+02	3.000000e+00
50%	6.914100e+04	2.167000e+03	3.500000e+00
75%	1.036370e+05	4.770000e+03	4.000000e+00
max	1.384930e+05	1.312620e+05	5.000000e+00

In [35]:
```
ratings['rating'].mean()
```

Out[35]:
```
np.float64(3.5255285642993797)
```

In [36]:
```
ratings.mean()
```

Out[36]:
```
userId     69045.872583
movieId     9041.567330
rating         3.525529
dtype: float64
```

In [37]:
```
ratings['rating'].min()
```

Out[37]:
```
0.5
```

In [38]:
```
ratings['rating'].max()
```

Out[38]:
```
5.0
```

In [39]:
```
ratings['rating'].std()
```

Out[39]:
```
1.051988919275684
```

In [40]:
```
ratings['rating'].var()
```

Out[40]:
```
1.1066806862788214
```

In [41]:
```
ratings['rating'].mode()
```

Out[41]:
```
0    4.0
Name: rating, dtype: float64
```

In [42]:
```
ratings.corr()
```

Out[42]:

	userId	movieId	rating
userId	1.000000	-0.000850	0.001175
movieId	-0.000850	1.000000	0.002606
rating	0.001175	0.002606	1.000000

In [48]:
```
filter1 =ratings ['rating'] > 10
print(filter1)
filter1.any()
```

```
0           False
1           False
2           False
3           False
4           False
            ...
20000258    False
20000259    False
20000260    False
20000261    False
20000262    False
Name: rating, Length: 20000263, dtype: bool
```

Out[48]: np.False_

In [50]:
```
filter2 = ratings['rating'] > 0
print(filter2)
filter2.any()
```

```
0           True
1           True
2           True
3           True
4           True
            ...
20000258    True
20000259    True
20000260    True
20000261    True
20000262    True
Name: rating, Length: 20000263, dtype: bool
```

Out[50]: np.True_

Data Cleaning handling

In [52]:
```
movies.shape
```

Out[52]: (27278, 3)

In [58]:
```
movies.isnull
```

Out[58]:
```
<bound method DataFrame.isnull of        movieId                               title  \
0            1                    Toy Story (1995)
1            2                      Jumanji (1995)
2            3             Grumpier Old Men (1995)
3            4            Waiting to Exhale (1995)
4            5  Father of the Bride Part II (1995)
...        ...                                   ...
27273   131254        Kein Bund für's Leben (2007)
27274   131256       Feuer, Eis & Dosenbier (2002)
27275   131258                  The Pirates (2014)
27276   131260                 Rentun Ruusu (2001)
27277   131262                    Innocence (2014)

                                       genres
0      Adventure|Animation|Children|Comedy|Fantasy
1                   Adventure|Children|Fantasy
2                              Comedy|Romance
3                        Comedy|Drama|Romance
4                                      Comedy
...                                       ...
27273                                  Comedy
27274                                  Comedy
27275                               Adventure
27276                   (no genres listed)
27277                 Adventure|Fantasy|Horror

[27278 rows x 3 columns]>
```

In [66]:
```
movies.isnull().any().any()
```

Out[66]:
```
np.False_
```

In [63]:
```
ratings.isnull().any().any()
```

Out[63]:
```
np.False_
```

In [67]:
```
tags.shape
```

Out[67]:
```
(465564, 4)
```

In [68]:
```
tags.isnull().any().any()
```

Out[68]:
```
np.True_
```

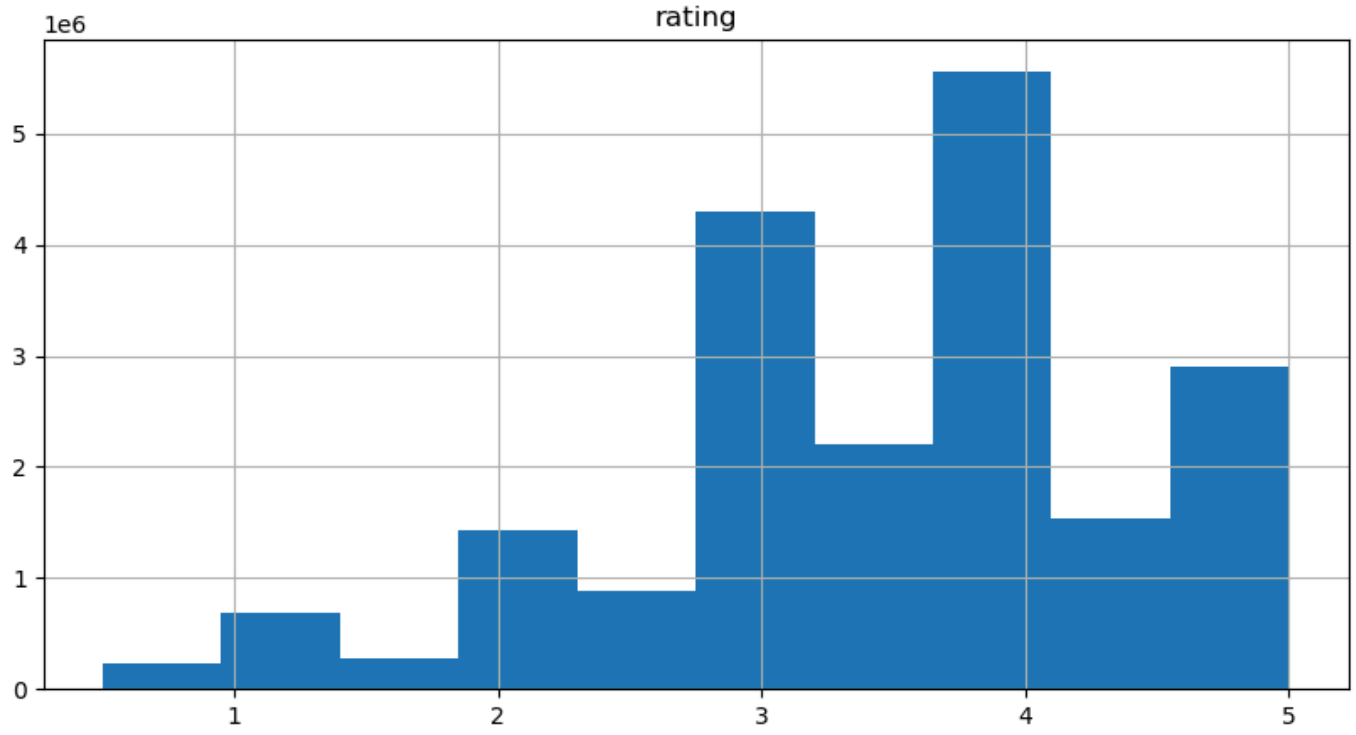
In [69]:
```
tags.shape
```

Out[69]:
```
(465564, 4)
```

Data Visualization

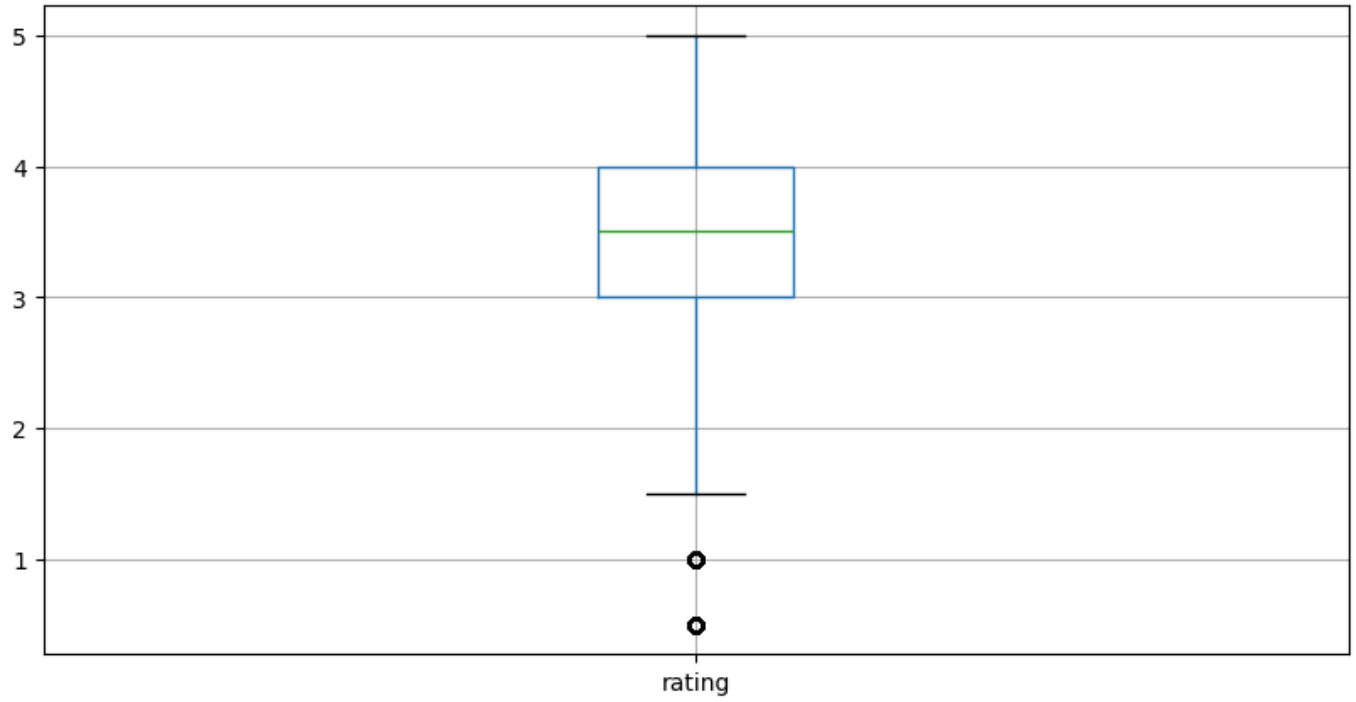
In [72]:
```
%matplotlib inline
ratings.hist(column ='rating', figsize = (10,5))
```

Out[72]:
```
array([[<Axes: title={'center': 'rating'}>]], dtype=object)
```



In [73]:
```
ratings.boxplot(column = 'rating', figsize = (10,5))
```

Out[73]: <Axes: >



Slicing Out Columns

In [74]:
```
tags['tag'].head()
```

Out[74]:
```
0      Mark Waters
1        dark hero
2        dark hero
3    noir thriller
4        dark hero
Name: tag, dtype: object
```

In [76]:
```
movies[['title','genres']].head()
```

Out[76]:

	title	genres
0	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	Jumanji (1995)	Adventure|Children|Fantasy
2	Grumpier Old Men (1995)	Comedy|Romance
3	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	Father of the Bride Part II (1995)	Comedy

In [78]:
```
ratings[-10:]
```

Out[78]:

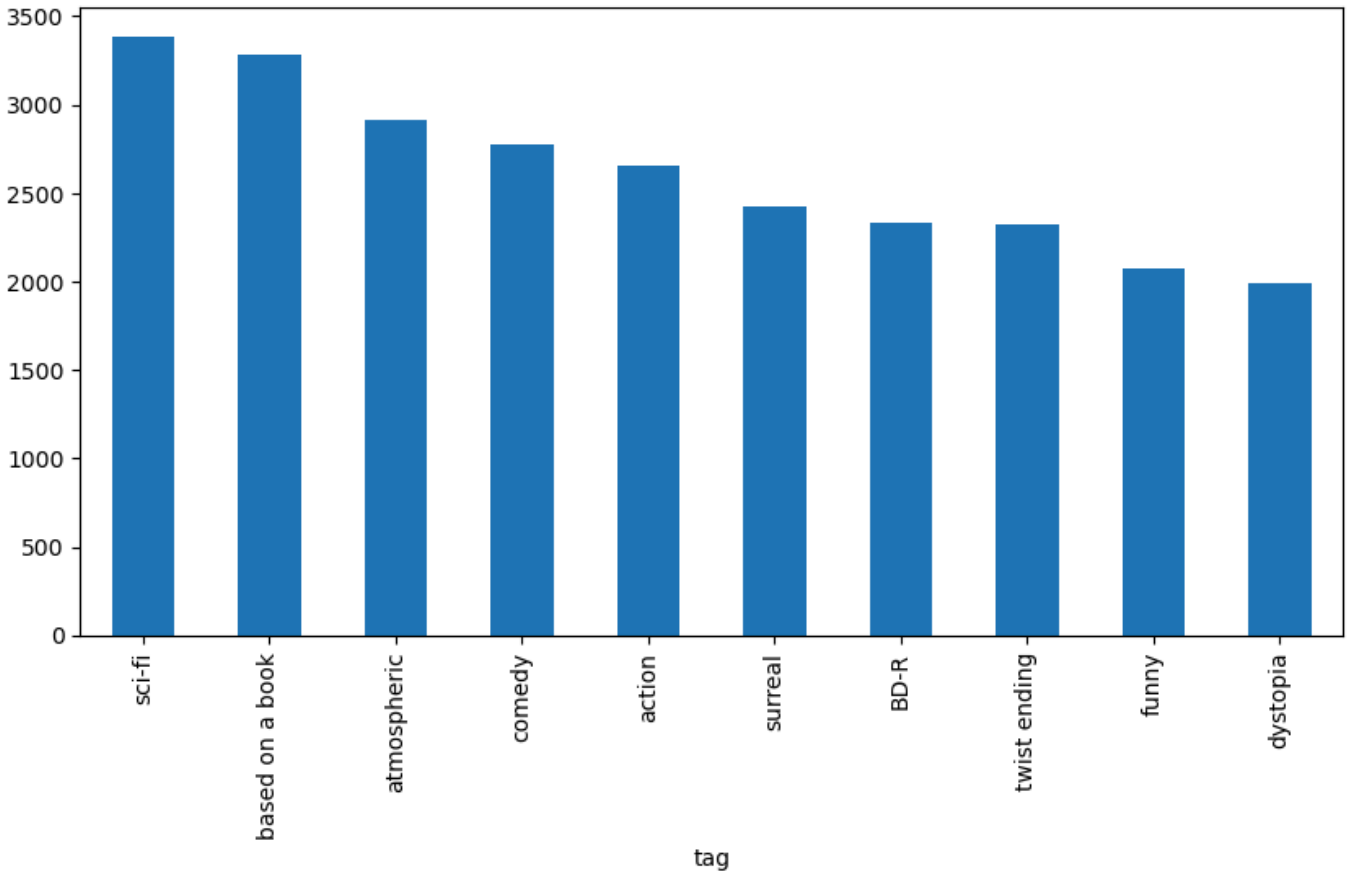
	userId	movieId	rating
20000253	138493	60816	4.5
20000254	138493	61160	4.0
20000255	138493	65682	4.5
20000256	138493	66762	4.5
20000257	138493	68319	4.5
20000258	138493	68954	4.5
20000259	138493	69526	4.5
20000260	138493	69644	3.0
20000261	138493	70286	5.0
20000262	138493	71619	2.5

In [81]:
```
tag_count = tags['tag'].value_counts()
tag_count[-10:]
```

Out[81]:
```
tag
missing child                   1
Ron Moore                       1
Citizen Kane                    1
mullet                          1
biker gang                      1
Paul Adelstein                  1
the wig                         1
killer fish                     1
genetically modified monsters   1
topless scene                   1
Name: count, dtype: int64
```

In [88]:
```
tag_count[:10].plot(kind='bar',figsize=(10,5))
```

Out[88]:
```
<Axes: xlabel='tag'>
```


3500
3000
2500
2000
1500
1000
500
0
sci-fi
based on a book
atmospheric
comedy
action
surreal
BD-R
twist ending
funny
dystopia
tag

In [ ]: